

An experimental investigation of personality types impact on pair effectiveness in pair programming

Panagiotis Sfetsos · Ioannis Stamelos · Lefteris Angelis ·
Ignatios Deligiannis

Published online: 7 November 2008
© Springer Science + Business Media, LLC 2008
Editor: Laurie Williams

Abstract In this paper, pair programming is empirically investigated from the perspective of developer personalities and temperaments and how they affect pair effectiveness. A controlled experiment was conducted to investigate the impact of developer personalities and temperaments on communication, pair performance and pair viability-collaboration. The experiment involved 70 undergraduate students and the objective was to compare pairs of heterogeneous developer personalities and temperaments with pairs of homogeneous personalities and temperaments, in terms of *pair effectiveness*. Pair effectiveness is expressed in terms of *pair performance*, measured by communication, velocity, design correctness and passed acceptance tests, and *pair collaboration-viability* measured by developers' satisfaction, knowledge acquisition and participation. The results have shown that there is important difference between the two groups, indicating better communication, pair performance and pair collaboration-viability for the pairs with heterogeneous personalities and temperaments. In order to provide an objective assessment of the differences between the two groups of pairs, a number of statistical tests and stepwise Discriminant Analysis were used.

Keywords Pair programming · Personality/temperament type · Pair performance · Pair effectiveness · Pair communication · Pair collaboration-viability · Controlled experiment · KTS-test

P. Sfetsos (✉) · I. Stamelos · L. Angelis
Department of Informatics, Aristotle University, 54124 Thessaloniki, Greece
e-mail: sfetsos@it.teithe.gr

I. Stamelos
e-mail: stamelos@csd.auth.gr

L. Angelis
e-mail: lef@csd.auth.gr

I. Deligiannis
Department of Information Technology, Alexander Technological Education Institute, P.O. Box 141,
GR-57400 Thessaloniki, Greece
e-mail: igndel@it.teithe.gr

1 Introduction

Pair programming, as an intensely social and collaborative activity (Beck 2000; Cockburn and Williams 2000; Cockburn 2002; Williams and Kessler 2003), capitalizes on developer's unique skills, experiences, idiosyncrasies, and personalities. The impact of developer personalities on communication and collaboration has been recognized as a critical success factor (Beck 2000; Bradley and Hebert 1997; Cockburn 2002; Ferdinandi 1998; Highsmith 2002).

In pair programming, as in any software process, there exist human factors (including developer personalities and temperaments) that can not be easily identified and understood well enough to be controlled, predicted, or manipulated. These factors can cause problems in pair effectiveness and in pair performance and must be addressed successfully. Such problems are not addressable through the known improvement approaches, as most of them focus only on processes or technology, not on people. However, many organizations and managers still approach pair programming as a rough technical process (not caring about developer personalities and temperaments; Gallis et al. 2003; Macias 2004; Sfetsos et al. 2006).

We consider pairs as *adaptive ecosystems*, in which physical structures, roles, and developers with personalities all exert forces on each other. We base this approach on the theory and conclusions of research studies which focused on software teams and group dynamics (Cockburn 2002; Coleman 1990; Forsyth 2006; Kirkman et al. 2001; Leana and Van Buren 1999). Cockburn (2002) in his "Team Ecologies" describes teams as ecosystems composed of personalities from diverse cultures which affect team effectiveness and performance. The authors in (Curtis et al. 1988) investigated software teams from the perspective of various disciplines, including different factors as psychological factors, skills, and communication issues. Existing research work on group dynamics investigated the performance of heterogeneous groups and homogeneous groups (Forsyth 2006) and proposed the relationships between regular communication, informal interaction, and social capital (Coleman 1990; Leana and Van Buren 1999). Pairs are adaptive because through pair rotations, developers can create, learn and respond to change. In these miniature ecosystems the overall development activity becomes a joint effort, a function of how paired developers communicate, interact and collaborate to produce results. However, different personalities express different natural preferences on communication, information and knowledge handling and sharing, decision making, and problem solving (Cockburn 2002; Ferdinandi 1998; Highsmith 2002; Keirse 1998; Myers 1962). Different personalities can be more or less effective, more or less appropriate for different tasks and functional or pair roles. Cockburn claims that developers with different personalities and with the same experience, if effectively combined, can minimize the communication gap (Cockburn 2002). Existing work on group dynamics has also shown that heterogeneous groups perform better than homogeneous groups (Forsyth 2006).

Managers have much to gain from psychology to understand where and why slowdowns occur (Beck 2000; Cockburn 2002; Ferdinandi 1998; Gorla and Lam 2004). This means that they must utilize processes which first identify and understand developers' personalities and then capitalize on their potential talents and strengths, effectively combining them. Some recent empirical studies (Canfora et al. 2004; Chao and Atli 2006; Chong and Hurlbutt 2007; Katira et al. 2004, 2005; Lui and Chan 2006; Melnik and Maurer 2002, 2004; Thomas et al. 2003; Williams et al. 2006), empirically investigated the impact of human issues (such as developer skills, developer personalities,

developer compatibility issues, etc.) on pair programming (see Section 2). However, these effects have not yet been the subject of controlled experiments. There still exist questions on how to match developers' personality and temperament types to their potential roles and tasks on pair programming and on how developer personalities and temperaments affect communication and pair collaboration-viability. Thus, the research challenge is to acquire evidence related to the question: Do developer personalities and temperaments affect pair effectiveness and more specifically do heterogeneous developer personalities and temperaments affect pair effectiveness and especially communication, collaboration and pair viability?

To (a) answer the above stated research question, (b) investigate Cockburn's claims on communication gaps and (c) to investigate Forsyth's findings for better performance of the heterogeneous groups we conducted a formal experiment. The objective of the experiment was to compare pairs of heterogeneous developer personalities and temperaments with pairs of homogeneous personalities and temperaments, in terms of pair effectiveness. Pair effectiveness is based on the same theoretical background as team/group effectiveness (Forsyth 2006; Halfhill et al. 2005; Nielsen et al. 2005; Stevens 1998; Sundstrom et al. 1990). It is grounded on the same subconstructs and it is described with the same terms as those used in team/group effectiveness, including communication and collaboration-viability variables. The definitions used fit also the terminology of studies addressing the problems of team productivity and effectiveness (Stevens 1998).

Seventy students attending a fourth semester SE course in the Department of Information Technology at the Alexander Technological Education Institute of Thessaloniki, Greece, participated in the experiment. The students were separated randomly into two groups of pairs according to their personality and temperament inventories. The Keirseley Temperament Sorter test (Keirseley and Bates 1984; Keirseley 1998) was used to identify and interpret students' personalities and temperaments in a separate session. Students in the main experiment and in the pilot run of the experiment procedure designed, coded and tested in Java two tasks on a well known experiment object, Cockburn's responsibility driven coffee machine design (Cockburn 1998).

The remainder of this paper is organized as follows. Section 2 summarizes related empirical work on pair programming. Section 3 presents the theory of personalities and temperaments and their identification and interpretation process. Section 4 describes the details of the experiment and students' inventories creation and the relative results. In Section 5 we present the analysis of the data, while in section 6 we present the interpretation of the results and we discuss the validity threats. Finally, we draw our conclusions and summarize our findings in Section 7.

2 Related Work

Earlier experiments have concluded that pair programming has many benefits over individual programming, including code correctness, reduced time to market, increased productivity and better product quality (Jensen 2003; Lui and Chan 2003; Müller 2003, 2005; Tomayko 2002; Williams et al. 2000, 2004; Williams 2001). However, some other experiments and empirical studies have reported contradictory results (Cockburn and Williams 2000; Hulkko and Abrahamsson 2005; Gallis et al. 2002; McDowell et al. 2002; Müller and Tichy 2001; Nawrocki and Wojciechowski 2001; Nosek 1998). To explain these differences in results, Williams et al. (2000) adduced anecdotal evidence supporting the

claim that pair programming needs an adjustment period called ‘pair jelling’. Other researchers explain these contradictions because of:

- differences in the context in which the studies were conducted (Arisholm et al. 2007; Gallis et al. 2003; Keefer 2005; Macias 2004)
- varying skills and experience, such as between novice–novice and expert–expert (Lui and Chan 2006)
- system complexity and programmer expertise (Arisholm et al. 2007).

We believe that most of these earlier empirical studies were based only on rough technological–economical factors, largely ignoring the human factors (developer skills, developer personalities and temperaments, etc.) involved. This one-sided consideration induced inadequacies in the subject selection, biased the results and illustrated a highly controversial and incomplete picture for pair programming.

In a family of newer empirical studies, especially those conducted in a university environment, where pair programming is used as a pedagogical teaching technique, human factors have been included in the experiments’ variables system. In a survey-based study (Melnik and Maurer 2002) reported that students preferred working with a partner who matched their own qualifications and experiences. Additionally, some of the stronger partners suggested that such pairs are more productive when the stronger of the pairs is the driver. Thomas et al. (2003), in another survey-based study, found that students with lower self-esteem liked pair programming more than students with higher self-esteem, while high self-esteem students disliked pair programming, mostly when paired with a low self-esteem partner. Additionally, students seemed to do their best work when paired with students of similar levels of self-esteem.

Canfora et al. (2004) in an experiment explore the hypothesis that working in pairs can speed up and enforce the knowledge building process among designers. The preliminary results confirmed the hypothesis of a positive effect of working in pairs on the process of knowledge building.

Melnik and Maurer (2004) in a repeated study investigated the role of conversation and social interactions as the key element of effective knowledge sharing in an agile process. Among the findings of this study is that verbal face-to-face interaction facilitates achieving higher velocity of software development teams.

Katira et al. (2004, 2005), in order to understand how some factors affect pair compatibility in students learning programming, conducted empirical studies focusing on self-esteem, skill level and personality. In these studies an important finding among other was that freshmen work better with partners of different Myers–Briggs personality type.

Williams et al. (2006) concluding the results of a two-phased empirical study, which lasted three years, reports that students of similar perceived technical competence are more likely to be compatible. Moreover, pairs comprised of sensor-intuitor learning styles seem to be compatible, while pairs with differing work ethic are not compatible.

Lui and Chan (2006) in an empirical study investigated the relationships between pairs of varying skills and experience, such as between novice–novice and expert–expert. Among the findings of the experiment was that novice–novice pairs against novice solos are much more productive than expert–expert pairs against expert solos.

Chao and Atli (2006) in an empirical study first surveyed the programmers in industry to identify the perceived important personality traits for pair programming, and then conducted experiments to determine the significance of these personality traits in successful pair programming. A preliminary result of this research showed that matching programmers based on their levels of the traits called ‘open-minded’ and ‘responsible’

could result in higher quality code. However, these differences were statistically not significant.

Chong and Hurlbutt (2007) in an ethnographic study of professional pair programming behavior on two software development teams found that pair programmers behave in ways inconsistent with the driver/navigator division of labor that is described in the pair programming literature. They identified expertise and keyboard control as important factors influencing pair programming interactions.

In this section we summarised the most important findings from empirical studies on pair programming, especially from those studies that include human factors in their research frameworks and models. There still exist many issues that have not been empirically investigated through a controlled experiment, such as the impact of developer personalities and temperaments on communication, collaboration-viability and in general performance and effectiveness in pair programming.

3 Identifying Personalities and Temperaments

Myers–Briggs type indicator (MBTI¹) is a 94-item questionnaire of preferences, which is used to identify quickly a person's psychological type, which can then be related to a temperament (Myers 1962). The MBTI focuses on four areas of behavior preferences, using pairs of opposing concepts (forming a total of 16 distinct psychological types) to identify where people *get their energy*, how they *gather information*, how they *make decisions*, and which *work style they prefer*. The identification and understanding of these behavior preferences can help us shape the different *communication styles*. The four pairs of opposite preferences are *extroverting* (E) and *introverting* (I), *sensing* (S) and *intuiting* (N), *thinking* (T) and *feeling* (F), and *judging* (J) and *perceiving* (P). Within each pair of opposite preferences, a person leans toward one or the other in most cases. Each individual will have some of the characteristics of the non-preferred attributes. No preference is better than its opposite (see Appendix 3).

David Keirsey, a behavioral scientist who developed the modern theory of temperament, correlated his theory into the MBTI system and classified the sixteen personality types into four temperament types (Keirsey and Bates 1984; Keirsey 1998). The four types were established when he found that by combining MBTI's sensing with the perceiving functions, (SP and SJ), and intuition with the judging functions, (NF and NT), he could express a person's preference for being expressive versus attentive, observant versus introspective, tough-minded versus friendly and scheduled versus probing. Thereby:

- *Artisan*—combines sensor, perceiver traits (SP)
- *Guardian*—combines sensor, judger traits (SJ)
- *Idealist*—combines intuitive, feeler traits (NF)
- *Rational*—combines intuitive, thinker traits (NT)

The Keirsey temperament sorter (KTS²) is a 70 question forced choice scored instrument with which temperaments can be quickly identified. It is available on the web (at www.keirsey.org) and the hardcopy is also available through the book “Please Understand Me”

¹ This work is funded by the Greek Ministry of Education (25%) and European Union (75%) under the EPEAK II program “Archimedes II”.

² Myers–Briggs type indicator and MBTI are registered trademarks of the Myers–Briggs Type Indicator Trust.

(Keirsey and Bates 1984). We used the hardcopy version of the KTS-test to identify and interpret the personality inventories of all participants in the experiment. The reason is that the KTS-test is shorter and cheaper than MBTI, provides the same information and runs quickly and easily (it takes about 10–15 min to answer the questions).

The scales of the KTS-test are as follows: for each pair of preferences, i.e. extrovert–introvert, extrovert is at the top of the scale, whereas introvert is at the bottom of the same scale. When scoring the KTS-test, the person ends up with an extrovert–introvert score; For the KTS-test, the extrovert–introvert scale ranges from 0 to 10 (0 indicates an extreme introvert and a 10 indicates an extreme extrovert), and the other three scales range from 0 to 20.

All personalities and temperaments have value, but we must create a comfortable setting for all types. By understanding developer variations and knowing what motivates them we can facilitate the pair formation and pair rotation process, allowing individuals to work in areas in which they are strong. Laplante and Neil (Laplante and Neil 2006) claim that: “Having understood people motivations, it becomes easier to seek win–win solutions or avoid causing problems”.

The results of both the MBTI-test and KTS-test, relying solely on self-report, should not be considered accurate to a full extent (Berens 1998). The results of these tests describe the individual preferences in a specific point of time and there always exists a risk that individuals report untrue preferences. Moreover, these tests are designed to measure a very small fraction of the human personality.

The Center for Applications of Psychological Type (CAPT 2006) website, in a section on ethical use, advises administrators to interpret the results conservatively because: “Type does not reflect an individual’s ability, intelligence, likelihood of success, emotions, or normalcy. Type is one important component of the complex human personality.” However, personality/temperament type assessment has proven to be effective as a tool for understanding individual differences and improving relationships (Captez 2003; Ferdinandi 1998; Gorla and Lam 2004; McDonald and Edwards 2007). In this experiment, we used the KTS-test because its results provide a first step towards understanding the broader dimensions of personality differences, making the classification of the groups more manageable (fewer combinations) than using the MBTI-test. Moreover, trait based instruments are the most appropriate in studies “where researchers wish to identify the personality factors related to software engineering, or those factors that typify a group of software engineers” (McDonald and Edwards 2007).

4 The Experiment

Our motivation for conducting the experiment is that the results can be used for the improvement of pair formations and rotations, improving the overall pair programming process. The experiment was executed in controlled setting with the purpose to control possible unwanted environmental factors, such as disturbances and noises from others, inappropriate work space, different tools, etc., or other organizational factors, such as disturbances from other tasks or jobs, fragmentation of the time subjects would dedicate to the tasks, direct measurement process, etc. We used students (instead of professional developers) as subjects because of the need for a controlled setting, a large number of participants and the one month time period needed for the preparation and execution of the experiment (see Sections 4.6 and 4.7). Controlled experiments with students have been characterized as formal experiments (Basili et al. 1999) and can provide confirmatory evidence, identifying interesting hypotheses for further investigation (Briand et al. 1997). In

the main experiments and in the pilot run of the experiment procedure emphasis has been given on two issues:

- The identification and interpretation of personalities and temperaments implemented through the procedure described above. The KTS test was run in a separate session, with the help of a psychologist from the Department of Medical and Health Care, who also helped us in the test preparation, in the interpretation of the results, and in the establishment of the personality and temperament lists. Before the test execution the students were informed on the type of questions they had to answer. Besides, they had been assured that the test results would not be published and would be used only in the pair formation and pair allocation process.
- The experiment was executed following strict planning, operation and analysis procedures, as proposed in the literature (Briand et al. 1999; Kitchenham et al. 2002; Wohlin et al. 2000), and will be analytically presented in the following sections.

4.1 Hypotheses Formulation

The *hypotheses* tested were the following:

(H₀₁): Heterogeneous developer personalities and temperaments do not affect pair performance.

(H₀₂): Heterogeneous developer personalities and temperaments do not affect pair collaboration-viability.

The statistical tests will attempt to reject the null hypotheses in favor of the alternative hypotheses, which are the opposite of H₀₁ and H₀₂.

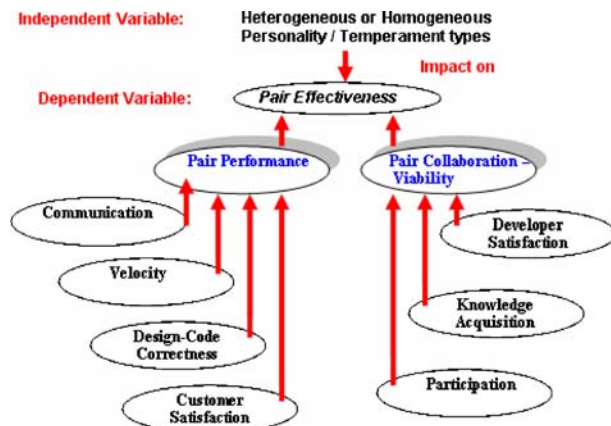
4.2 Variable Selection-Communication and Collaboration-Viability Metrics

The independent variable (factor) is the type of composition of the pairs of developers. It is of nominal type, with two possible values (see Fig. 1): (a) homogeneous personality and temperament types and (b) heterogeneous personality and temperament types. The dependent variable is the pair effectiveness which is measured by pair performance and pair collaboration-viability (see Table 1).

Communication metrics are classified as process metrics, because they measure collections of software-related activities within a process (Fenton 1991; Langley 1999). These metrics have been successfully used for studying the information flow in software projects (Dutoit and Bruegge 1998; Langley 1999; Saeki 1995; Seaman and Basili 1997). We used communication metrics, as process metrics, to compute the volume of the information exchanged among developers during pair programming (in relation to time), and to relate it to the overall performance. A restriction concerning this variable, in our study, is that only bidirectional communication could be measured. We provide some definitions in order to clarify the communication metrics we used.

1. *Communication modes* refer to different types of information exchange that have defined objectives and scope (e.g. face-to-face, e-mail, teleconference, etc). Communication modes are characterized as scheduled if communication is planned, or as event-driven if they occur occasionally.
2. *Communication mechanism* is a tool or procedure used to transmit and receive information and supports a communication mode.

Fig. 1 The variables system. Hypotheses are investigated for *highlighted* dependent variables



In our experiment we decided to use a face-to-face, synchronous transactive communication mechanism because both sender and receiver are available at the same time and transmit messages to one another simultaneously. Pair programming activities such as requirements gathering, specification and design changes, coding, unit testing, code and design reviews, are scheduled. Therefore, we decided to measure communication by recording the volume of those information exchanges, characterizing them as *communication transactions*. A communication transaction is defined as an information exchange between the driver and the navigator (and vice versa) in order to add, modify, delete, test, and review requirements, specifications, designs, code and unit tests (see Table 1 and

Table 1 Goals, questions and metrics for the experiment

Goal	Question	Metrics
Evaluate pair effectiveness (performance and collaboration-viability) in heterogeneous and homogeneous pairs	Which factors have an impact on pair effectiveness?	Metrics for performance variables
	More specifically:	
	Which factors have an impact on pair performance?	•Time in minutes to finish assignments •Points received for correct solutions •# of passed acceptance tests •# of transactions on communication modes including: ✓Requirements gathering trans. ✓Specification and Design changes trans. ✓Code trans. ✓Unit test trans.
	Which factors have an impact on pair collaboration-Viability?	✓Peer reviewing trans.
		Metrics for collaboration-viability variables
		•Collaboration satisfaction •Knowledge acquisition •Participation (degree of communication satisfaction, degree of nuisance, driver or navigator preference)

Appendix 1). The data for the communication variable (sum of all communication transactions of each pair) are derived from the communication records in specially designed transactions forms. Navigators were responsible for the recording process, checking in the number of messages exchanged for each of the predefined communication transactions (for details see Section 4.5).

Velocity was measured by the time needed for the completion of each task (finish time—start time) in minutes. Design correctness was measured by the points obtained by each pair for both tasks. The acceptance tests, of black-box type, were written in spreadsheets, and ran manually by the authors, after the students completed and delivered their code. Acceptance tests were chosen to test the business value of the students' designs and code and were not revealed to the subjects.

Pair collaboration-viability was measured by data gathered from a questionnaire, filled by both pair participants, after the code completion. Questions were derived from empirical research studies on team effectiveness (Stevens 1998; Sundstrom et al. 1990) and concerned:

1. Members' collaboration satisfaction,
2. Knowledge acquisition, a measure of learning effectiveness (Stoyanova and Kommers 2002; Khamesan and Hammond 2004; Ahmad and Hammond 2004). It reflects the degree of knowledge which has been elicited through pair programming,
3. Participation:
 - 3.1 Degree of communication satisfaction for each partner (level of communication satisfaction during pair programming). It reflects the communication feeling at the personal level.
 - 3.2 Degree of nuisance [external noises and disturbances (from workspace—other pairs)]. It reflects the degree of external disturbances.
 - 3.3 Voluntary or mandatory preference and driver or navigator preference.

4.3 Experimental Setting—Subject Selection

The experiment was performed at the Alexander Technological Educational Institution of Thessaloniki-Greece, Department of Informatics, during the SE course of the fourth semester. The course integrates agile software development processes and software measurement techniques performed by developers. The weekly lab sessions (duration 2 h) included the development of different projects by the students working in pairs, using different Java tools and applying test-first development. The laboratory lectures, home assignments and the final project were mandatory for the students, while the evaluation of the course was based on a pass/fail basis.

Seventy students (out of a total of 90) participated voluntarily in the experiment. Students were motivated to practice their “real-world” skills and gain educational benefits from their participation. Moreover, for participating they would gain a fixed bonus (10%) on their final assessment grade, but only after passing their final assessment examinations. Before the execution of the experiment students were neither aware of what we intended to investigate, nor were they informed about the actual hypothesis examined. However, after the experiment execution and the analysis of the results the whole experimentation process was explained analytically to the students.

The participating students had the proper level of knowledge and experience in pair programming as they have already been taught agile methods, extreme programming and

used pair programming in their laboratory assignments in the third semester during the course of Methodologies. We asked the students about any professional experience they already had. Four students with professional experience were excluded from the experiment, so all students had roughly the same expertise.

Students were divided randomly into two groups of pairs according to their personality/temperament inventories. We picked randomly names from the student list and formed pairs, allocating them either to the *control group* (homogeneous/same personalities/temperaments) or to the *experimental group* (heterogeneous/mixed personalities/temperaments). The total number of pairs was 17 pairs for the control group and 18 pairs for the experimental group (see the distribution of pairs in Section 4.7). Extroverts and introverts were equally scattered within the two groups. All other criteria such as gender and race were allowed to vary randomly. Two foreign students were allocated in different groups, and gender distribution resulted to be homogeneous among the two groups.

4.4 Definition

The goal-question-metric (GQM) template, described by Basili and Weiss (1984) and Basili and Rombach (1988), helped us to determine which metrics were appropriate for evaluating pair effectiveness. The formal definition (Briand et al. 1997) for the experiment is the following:

<i>Analyze</i>	Heterogeneous vs. homogeneous team composition for pair programming
<i>For the purpose of</i>	Evaluation
<i>With respect to</i>	Pair effectiveness (pair performance including communication and pair collaboration-viability)
<i>From the point of view of</i>	Organizations, managers, developers and researchers
<i>In the context of</i>	Undergraduate SE course, fourth semester, at the Department of Informatics of the Alexander Technological Educational Institution of Thessaloniki-Greece

Heterogeneous is used here to characterize teams (sets of pairs) or individual pairs with mixed personality and temperament types, while homogeneous refers to pairs with same types. The effectiveness of a pair can be described in terms of two aspects, pair performance and pair collaboration-viability. For the purposes of this experiment:

- *pair performance* consists of factors such as design and code correctness, velocity, communication, and passed acceptance tests (see Fig. 1)
- *pair collaboration-viability* consists of factors such as collaboration satisfaction, knowledge acquisition and participation (degree of communication satisfaction, degree of nuisance and driver or navigator preference).

4.5 Experimental Design and Instrumentation

We designed a between-subject experiment, which means that the comparison is between different groups of subjects (experimental and control group) for each level of the independent variable (treatment). Therefore, the assignments should be almost identical, providing communication occasions (modes) to the pair partners. Two maintenance tasks on Cockburn's responsibility driven coffee machine designs were the assignments that students had to design, code and test in Java in a time period of two and a half hours. We chose these two tasks because they represent a frequently used experimental object (discussed at a

workshop on object-oriented design quality at OOPSLA'97) and because we wanted to avoid mono-operation bias threat (see Section 6.2.3).

The first task concerned cash administration that starts with a cash amount, makes transactions, gives change and at the end prints the final amount. The second task had to do with statistics, namely log each sale transaction and eventually print in a descending order the most sold coffee types. Both tasks were about the same code size and difficulty level, thus we had a common baseline for comparing not only the designs and code, but also the number of communication transactions made during the pair programming session. We considered the two tasks to be of equivalent size based on the fact that in both cases four to five new methods and one new class should be implemented. Also, tasks resulted to be of approximately equal difficulty during the pilot run of the experiment procedure. The duration for both assignments was planned to last about two hours, but we expanded it for another half hour in order to ensure that students would have enough time to complete the assignments. Design and code correctness were assigned to each pair for both assignments and were measured on a ratio scale, in an interval from 0 (min) to 10 (max), based on checklists to ensure objective assessment of the participants' design and code solutions (see Appendix 4).

Communication transactions recording Early in the planning phase we examined all the available methods for transactions recording. Firstly, we examined the possibility to video and audio record the transactions, but this idea was rejected, considered to be costly and almost not applicable with so many pairs placed close to each other. Secondly, we examined if the already known automatic metrics collection tools such as the Empirical Project Monitor (Ohira et al. 2004) or plug-ins such as Hackystat (Johnson et al. 2003) or PROM (Sillitti et al. 2003) could be used to record the communication transactions. We found that those tools could not record all types of the information exchanges we needed in our experiment. Therefore, we decided that communication transactions should be recorded manually in already printed forms (see Appendix 1) by the navigator of each pair through a log keeping process, as in the personal software process³ (with differences in the types of the recorded data; Humphrey 1996, 1997). This decision was also supported by our consideration that the involvement of students in a measurement activity would make them understand the measurement process and would minimize their resistance to metrics. Navigators were asked to record the start and finish time for each task and tick the appropriate communication transaction each time the partners communicated. To ensure that the data collection process would be successful in the experiment we took the following actions (see also Section 6.2.1):

- Communication issues such as classification methods of communication transactions, especially in pair programming activities, have been taught in the regular course sessions and students practiced them during the laboratory sessions.
- The entire recording process was tested in a separate training session (see next section) and in a pilot run of the experiment procedure.
- The recording process was constantly monitored by the teachers during the experiment (see Section 4.7)

³ See <http://keirse.com/cgi-bin/keirse/kcs.cgi>.

Data gathered by the questionnaire, concerning collaboration-pair viability, were measured on an ordinal scale, from 1 (min) to 5 (max).

4.6 Preparation

To successfully conduct the experiment we had to prepare the subjects, material and instrumentation used, data collection forms, and the starting (kickoff) procedures. We designed the communication transactions forms and questionnaire as simple as possible, helping students to fill them easily. Moreover, we created in the local network a directory in which the communication transactions forms, questionnaires and explanation notes in form of text files were made available to the students. All pairs had access to their own directory in which they would save their code and the associated unit tests during the experiment. Finally, we planned and executed:

1. An informatory session in which all the items the students needed to know to successfully participate in the experiment were explained to them in sufficient detail
2. A pilot run of the experiment procedure in which the experimental design and the manual data collection procedures were tested. This pilot run of the experiment procedure was run a week before the main experiment, using the same hypotheses and different students as subjects.

In the training session the material used in the experiment (KTS-test, communication transactions forms, questionnaires) and the associated instrumentation were explained in detail. More specifically, students were sufficiently informed about:

- the entire experimental process, and the time schedules for the KTS session and the experiment
- the roles they should assume (driver-navigator) in each assignment. Drivers and navigators should switch roles after the completion of the first assignment (about 1 h and 15 min).
- the additional navigators' task and role in the experiment
- the communication transactions, and how to keep note of the proper columns when they communicated during the experiment
- the type of questions they had to answer in the questionnaire concerning collaboration-viability
- the ordinal scales they had to use in the questionnaire
- the KTS test and the type of questions it includes.

The KTS-test was run a week after the training session. In this session we identified and interpreted students' personalities and temperaments, establishing the personality and temperament inventories-the lists which we used in pair forming and pair allocating process.

4.7 Execution—Data Validation

After careful preparation, testing and improvement of the experimental material and the procedures concerning data collection, pair formation and pair allocation, we ran the experiment. The pair formation and pair allocation process took place half an hour before the beginning of the experiment. The distribution of the four temperament types, for the seventy students who participated in the experiment, is shown in Table 2, while the distribution of pairs in the control and experimental group is shown in Table 3.

Table 2 Distribution of temperament types for students

Temperament type	No. of students	Percent (%)
Artisan (SP)	7	10.0
Guardian (SJ)	42	60.0
Rational (NT)	6	8.57
Idealist (NF)	15	21.42

Two of the temperament types, artisans(SP) and rationals(NT), were minorities in our sample (10% and 8.6%, respectively) and this fact did not permit us to form pairs with all possible temperament types' combinations in both groups (see Section 6.2.3—confounding constructs and levels of constructs).

Before the experiment execution, a set of hard copies containing the transactions forms, the guidelines and two questionnaires (for both students) were provided to each pair. We used three large laboratory classrooms with 25 work stations in each and enough space between the computer stations to minimize the undesirable effects of disturbance and noises. Pairs were allocated randomly in the three classrooms, had their own computer station and saved their code and the associated unit tests in their private directories. During the execution of the experiment we carried out a strict control process in order to ensure that the navigators would fill correctly the forms. We closely observed the form filling process, setting two teachers with two assistants in each classroom, advising and supporting navigators in regular time intervals. Eventually, the data collection process was completed without problems.

Data for the two pair performance variables, namely communication transactions and time, were collected via the communication transaction forms. Students' performance on design/code/unit test correctness and passed acceptance tests was measured in terms of the checklist-based assessment of the task outcomes and the percentages of the passed acceptance tests respectively. Data for pair collaboration-viability were gathered through questionnaires. After the collection, all data were scrutinized to check whether they were generated correctly. We examined the accuracy, consistency, correctness and completeness of the data and found that they conformed to our expectations (see Section 5.1.3). After the students delivered the filled transactions forms and questionnaires, we ran the acceptance tests.

Table 3 The control and experimental groups of pairs

Control group		Experimental group	
Temperament type	No. of pairs	Temperament type	No. of pairs
NF-NF	4	NF-SJ	6
SJ-SJ	13	NF-SP	1
		NT-SJ	5
		NT-SP	1
		SJ-SP	5
Total	17		18

5 Data Analysis and Results

This section contains the results of the statistical analysis on the raw data, and an interpretation of the results. For the statistical analysis of the data concerning performance variables we used a set of statistical analysis methods:

- Descriptive statistics for a general overview of the data
- Correlations between the variables to see how the performance variables are related
- Comparisons between all variables using different univariate statistical tests for testing differences of each metric between the control group and the experimental group
- Multivariate stepwise discriminant analysis (DA) for testing differences of all the metrics together between the control group and the experimental group.

The descriptive statistics subsection uses numerical measures and most important, graphical methods to visualize any differences between groups in the performance variables. The correlation subsection examines the association between the performance variables. In the univariate comparisons subsection each performance variable is considered separately and is tested for differences between the two groups. In Section 5.1.4 all the performance variables are considered simultaneously and they are tested for their discriminating power as a whole and not separately.

Collaboration-viability data were analysed only quantitatively, through descriptive statistics and chi-square tests. Collaboration-viability in pair programming needs more exhaustive quantitative and qualitative research with questionnaires and in depth interviews.

5.1 Analysis of Pair Performance

5.1.1 Descriptive Statistics

The distribution (mean, median, minimum and maximum) and variance (standard deviation) of all variables separately for both tasks and groups and also for the totals of the variables indicate important differences between the two groups. Performance variables aggregating measurements from both tasks are given in Table 4. The box-plots in Fig. 2a–d,

Table 4 Descriptive statistics for the totals of the variables time, communication transactions, correctness points and acceptance tests for both tasks

Variable (both tasks)	Personality type group	N	Mean	Median	SD	Min	Max
Total time for two tasks	Control	17	100.00	100.00	15.116	75	135
	Experimental	18	74.28	75.00	12.924	45	95
	Total	35	86.77	83.00	19.005	45	135
Total communications for two tasks	Control	17	28.88	25.00	13.513	12	62
	Experimental	18	49.11	44.00	14.816	30	93
	Total	35	39.29	40.00	17.347	12	93
Total points for two tasks	Control	17	3.41	3.00	1.278	2	6
	Experimental	18	5.89	5.50	1.676	2	9
	Total	35	4.69	5.00	1.937	2	9
Total passed acceptance tests for two tasks	Control	17	91.76	100.00	40.348	30	160
	Experimental	18	147.22	150.00	32.504	60	190
	Total	35	120.29	130.00	45.664	30	190

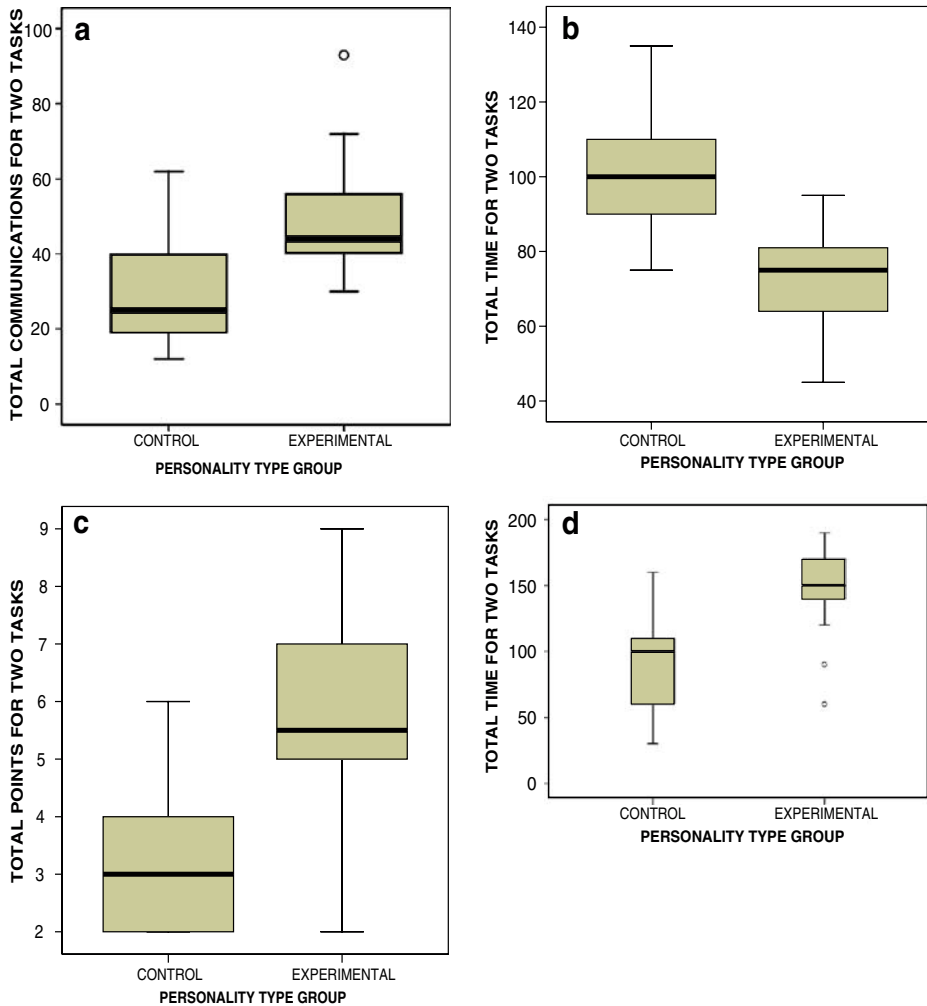


Fig. 2 Distribution of aggregated variables for both tasks in groups

show the distributions of the four aggregated variables (combination of statistics for both tasks) in the experimental and the control group. The box-plot is an informative way to represent the entire distribution of a variable. Specifically, the height of the box corresponds to the inter-quartile range (the difference between the 75% and the 25% quartiles of the distribution), the solid horizontal line represents the median and the two vertical lines (the whiskers) are extended to the minimum and maximum values in the data that are no outliers. Outliers and extreme points (points outside pre-specified ranges) are depicted with different type of points.

The visual inspection of the aggregated variables' statistics for both tasks, with the aid of box-plots, depicts important differences between the two groups. Specifically, the total communication transactions, the total points for design and code correctness and the aggregated acceptance tests for both tasks are significantly higher for the experimental group. The total time spent (i.e. velocity) for both tasks are significantly less for the experimental group.

In order to explore graphically the differences between the two groups in terms of more than two variables, we chose three basic performance variables, the total communication transactions, the time spent and the points for design and code correctness. These variables were used to produce scatter plots for representing the pairs comprising the two groups in two-dimensional spaces (Fig. 3a–c) and in three-dimensional spaces (Fig. 4) where the axes stand for two or three performance variables.

In each scatter plot the groups are represented by different markers. For each group a centroid point (i.e. the center of the group having as coordinates the mean values of the variables) is computed and the spikes from each point to the centroid are used to show the discrimination and the dispersion of the groups along with the relationship between the two variables. From all scatter plots we can see a clear discrimination of the two groups. In general, the experimental group is characterized by more

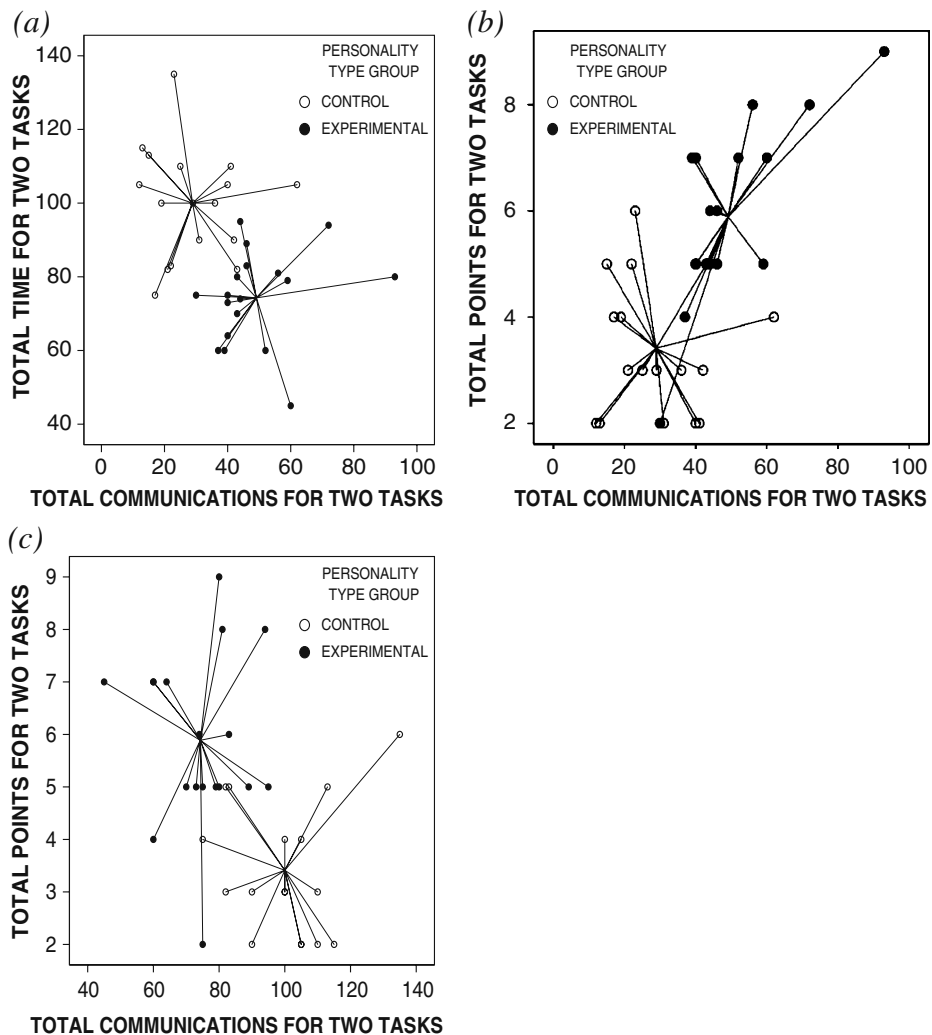
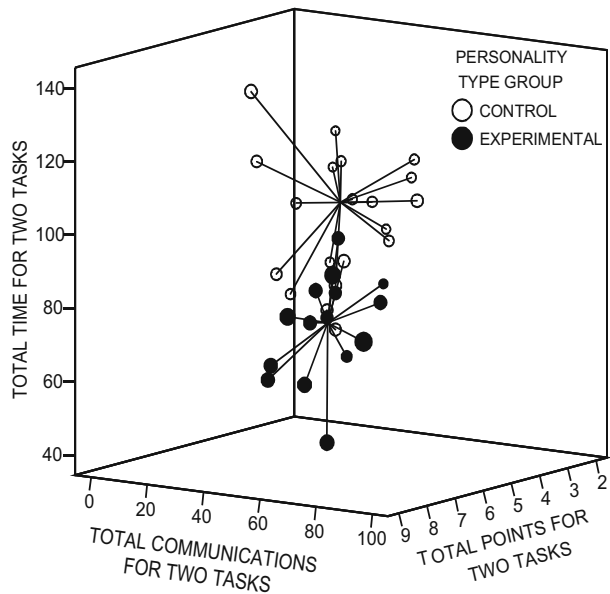


Fig. 3 Relationships between the performance variables for both tasks

Fig. 4 Three-dimensional scatter plots for both tasks



communication transaction, higher points in design and code correctness and less time spent than the control group.

5.1.2 Correlations

In this subsection we present the results of a correlation analysis conducted on the three basic performance variables measured from the experiment (time, communication transactions and points). Table 5 contains the nonparametric Spearman correlation coefficients for every pair of variables, separately for each group (control and experimental).

The coefficients are given for each task separately and for both tasks. The values of the Spearman correlation coefficient range from -1 to 1 and its sign shows the nature of the correlation, i.e. whether it is negative or positive. For each one of the coefficients a test for its significance was performed, so if a coefficient shows significant correlation (is significantly different than 0), it is accompanied by one or two stars (* and **) which show significance at the 0.05 and 0.01 level respectively. The Spearman coefficient is based on the ranks of the data rather than the actual values. That is why the coefficient is not affected by the existing outliers.

From Table 5 we can see that for the first task there is a significant positive correlation between communication transactions and design and code correctness only for the experimental group. This shows that, for the first task, the experimental group performs better (increased design and code correctness) as the communication transactions are increased while for the control group the communications do not seem to affect correctness. In contrast, the increased time spent is related (though not significantly) with negative results on the correctness of the control group.

Regarding the second task, we also see that there is an even more significant positive correlation between communication transactions and points, for the experimental group. So, we can infer again that the experimental group performs better as

Table 5 The Spearman correlation coefficients for each and both groups and tasks

Group	Task	Task 1		Task 2		Both tasks	
		Comm.	Points	Comm.	Points	Comm.	Points
Control	Time	+0.113	−0.404	−0.249	+0.057	−0.164	−0.139
	Comm.		+0.119		+0.039		+0.000
Experimental	Time	+0.009	+0.016	+0.283	+0.371	+0.374	+0.024
	Comm.		+0.476*		+0.687**		+0.694**

*Correlation is significant at the 0.05 level

**Correlation is significant at the 0.01 level

the communication transactions are increased while for the control group this is not true.

Considering both tasks together, we see that the aggregated time spent for both tasks is negatively correlated with communication transactions and points for the control group, though the correlation is not significant. However, the clearest conclusion comes from the positive correlation between communication transactions and correctness in the experimental group.

5.1.3 Comparisons—Univariate Statistical Tests

In this subsection we provide the results of univariate statistical tests which compare the values of each variable between the control and the experimental group. Specifically, we use two well-known statistical tests: The Student's t test and the non parametric Mann–Whitney (M–W) test. Table 6 gives the results of the comparisons for all variables between the two groups concerning the first and the second task. Table 7 contains the comparisons of the three basic performance aggregated variables for both tasks.

Each table contains, in different columns, the significances of the t test and of the M–W test. The results show that almost all variables have significant differences between the control and the experimental group, with only exceptions the transactions for design and code review that do not differ significantly for each of the tasks.

5.1.4 Discriminant Analysis (DA)

In Section 5.1.3, we saw that each one of the performance variables (time, communications and points) for each of the tasks are significantly different between the two groups (control and experimental). In order to investigate the six variables all together we test how well these variables are able to discriminate the pairs of the different groups. For this reason we use a multivariate technique known as stepwise discriminant analysis (DA). This method builds a linear model with the type of group as the dependent variable, and the six performance measurements for the first and the second task as the independent variables. Due to the fact that there are significant correlations between these variables, not all are necessary for an efficient model. This is handled by the stepwise procedure which selects the best variables for the model. The stepwise DA resulted in the following model with four out of six original variables:

$$Z = -2.963 + 0.062 * (\text{time spent for first task}) + 0.060 * (\text{Time spent for second task}) \\ - 0.067 * (\text{communication transactions for second task}) - 0.473 * (\text{points for second task})$$

Table 6 Comparisons of all variables between the two groups concerning first and second task

Variable	First task significance				Second task significance			
	Group	Mean	<i>t</i> test	M–W test	Group	Mean	<i>t</i> test	M–W test
Transactions for requirements gathering	Control	2.35	0.004**	0.006**	Control	2.06	0.004**	0.007**
	Experimental	4.22			Experimental	4.22		
Transactions for specifications and design changes	Control	2.88	0.054*	0.018*	Control	2.12	0.021*	0.002**
	Experimental	4.44			Experimental	3.72		
Transactions for coding	Control	5.47	0.000**	0.001**	Control	4.65	0.000**	0.000**
	Experimental	10.17			Experimental	9.89		
Unit tests	Control	2.35	0.015*	0.011*	Control	2.00	0.004**	0.002**
	Experimental	3.28			Experimental	3.39		
Transactions for design and code review	Control	3.06	0.948	0.673	Control	1.94	0.229	0.347
	Experimental	3.11			Experimental	2.67		
Time spent	Control	55.76	0.000**	0.000**	Control	44.24	0.003**	0.005**
	Experimental	41.61			Experimental	32.67		
Communication transactions	Control	16.12	0.002**	0.003**	Control	12.76	0.000**	0.000**
	Experimental	25.22			Experimental	23.89		
Points	Control	1.82	0.013*	0.023*	Control	1.59	0.001**	0.001**
	Experimental	2.83			Experimental	3.06		
Acceptance tests	Control	48.24	0.006**	0.006**	Control	43.53	0.000**	0.001**
	Experimental	72.22			Experimental	75.00		

*Significant difference at the 0.05 level

**Significant difference

Having the performance values of a pair, we can substitute them in the model and classify the pair to the control group if the score *Z* is positive or to the experimental group if the score is negative. The coefficients of the model show that larger values of the times spent favor the control group while larger values of communications and correctness points of the second task favor the experimental group. These findings are in accordance to the

Table 7 Comparisons of the four aggregated performance variables

Variable (both tasks)	Significance			
	Group	Mean	<i>t</i> test	M–W test
Total time	Control	100.00	0.000**	0.000**
	Experimental	74.28		
Total communication transactions	Control	28.88	0.000**	0.000**
	Experimental	49.11		
Total points	Control	3.41	0.000**	0.000**
	Experimental	5.89		
Total passed acceptance tests	Control	91.76	0.000**	0.000**
	Experimental	147.22		

*Significant difference at the 0.05 level

**Significant difference at the 0.01 level

general results of the analysis. The pairs of the control group are classified around a centroid, a point with mean score 1.551, while the pairs of the experimental group have a centroid with mean score -1.465 .

The model fitting accuracy can be measured by its ability to classify correctly the cases of the dataset. As can be seen from Table 8, the model can classify correctly 97.1% of the cases in the two groups. The same Table contains the results of a cross-validation procedure which gives 91.4% correct classifications. This is a measure of the model's predictive accuracy. Each time a case is removed from the data, a model is constructed on the remaining cases and the removed case is then classified to a group according to the rule obtained by the model. Specifically, only one of the 17 control pairs is misclassified as experimental pairs while two of the 18 experimental pairs are misclassified as control pairs. Generally, the discrimination is very good, since the correct classification rate is much higher than that of a completely random allocation to groups which would have been around 50%. In fact, it can easily be seen that a simple z -test for testing the null hypothesis that the correct classification proportion does not differ significantly from 0.5 ($H_0: p=0.5$) is rejected at level 0.05. This result shows that the overall behavior and performance of the two groups is significantly different.

5.2 Questionnaire Results: Pair Collaboration-Viability

Data gathered from questionnaires concern developers' collaboration satisfaction, knowledge acquisition and participation (degree of communication satisfaction, nuisance ratio, and driver or navigator preference). Data were analysed quantitatively using descriptive statistics (contingency tables and bar charts) and the chi-square test. Specifically, the answers to the questions were originally ratings in a 1–5 ordinal scale. In order to examine better the trends in the preferences and draw clear conclusions, we combined, in each of the questions, the levels 1, 2 and 3 into a new category, labelled “neutral or negative” attitude and the levels 4 and 5 into another category, labelled “positive” attitude. The new categories were cross-tabulated with the two groups and the chi-square test was used to infer whether there was significant difference in the attitudes between groups.

Table 8 Classification and cross-validation results

			Predicted group membership		Total
			Control	Experimental	
Original	Count	Control	16	1	17
		Experimental	0	18	18
	Percentage	Control	94.1	5.9	100.0
		Experimental	0.0	100.0	100.0
Cross-validated(a)	Count	Control	16	1	17
		Experimental	2	16	18
	Percentage	Control	94.1	5.9	100.0
		Experimental	11.1	88.9	100.0

97.1% of original grouped cases correctly classified. 91.4% of cross-validated grouped cases correctly classified

5.2.1 Developers' Collaboration Satisfaction

Q-1: *Using the ordinal scale 1 (min)-to-5 (max), please rate your collaboration satisfaction*

The chi-square test shows significant difference in the satisfaction between the two groups ($p=0.007$). This can be explained by observing that exactly half of the control group rated positively the collaboration satisfaction, while the majority of the experimental group responded positively. This can be seen clearly in the bar chart of Fig. 5.

5.2.2 Knowledge Acquisition

Q-2: *Using the ordinal scale 1 (min)-to-5 (max), please rate your knowledge acquisition*

The chi-square test shows significant difference ($p=0.025$) between the two groups in knowledge acquisition. Most of the pairs of the control group respond neutrally or negatively, while the majority in the experimental group responds positively (see also Fig. 6).

5.2.3 Participation: Degree of Communication Satisfaction

Q-3: *Using the ordinal scale 1 (min)-to-5 (max), please rate your communication satisfaction*

The chi-square test shows significant difference ($p=0.017$) between the two groups. This can be explained by the fact that most of the control group rate neutrally or negatively the communication while most of the experimental group respond positively to communication (see also Fig. 7).

5.2.4 Participation: Degree of Nuisance

Q-4: *Using the ordinal scale 1 (min)-to-5 (max), please rate the nuisance*

From chi-square test ($p=0.318$) and Fig. 8, it seems that there is no significant difference of the degree of nuisance between the two groups.

Fig. 5 Developers' satisfaction

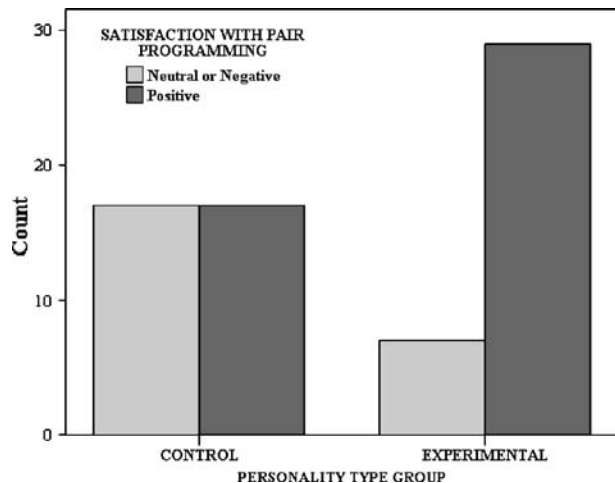
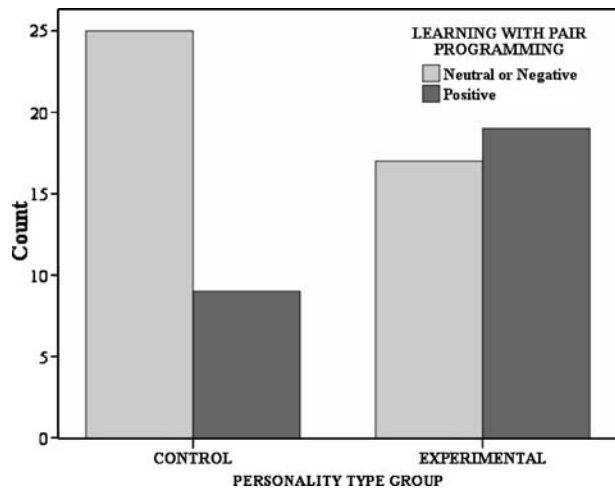


Fig. 6 Knowledge acquisition

5.2.5 Participation: Driver or Navigator Preference

Q-5: *Using the ordinal scale 1 (min)-to-5 (max), please rate your driver or navigator preference*

There is no evidence of difference of those who prefer coding between the two groups (chi-square test: $p=0.586$). Also, there is no significant difference for the reviewing preference since the responses are about the same (chi-square test: $p=0.268$; see Fig. 9). It is worth mentioning that everyone in the control group ranked the review positively.

In general, the preliminary results from the data analysis of the questionnaires revealed better pair collaboration and viability for the group with heterogeneous types. In particular,

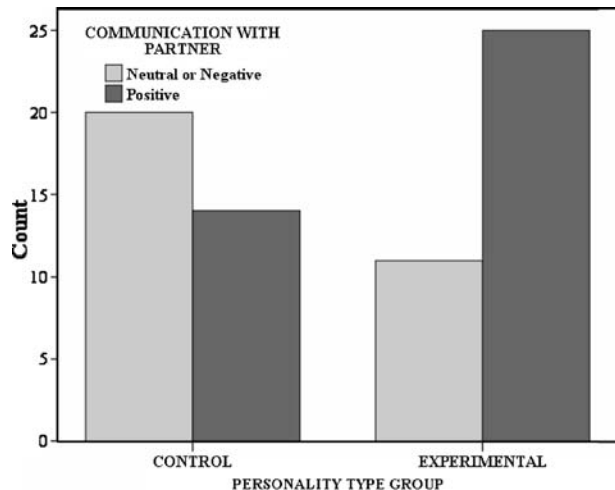
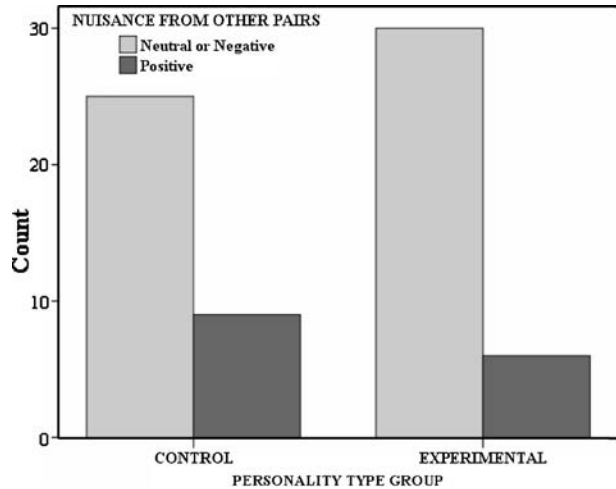
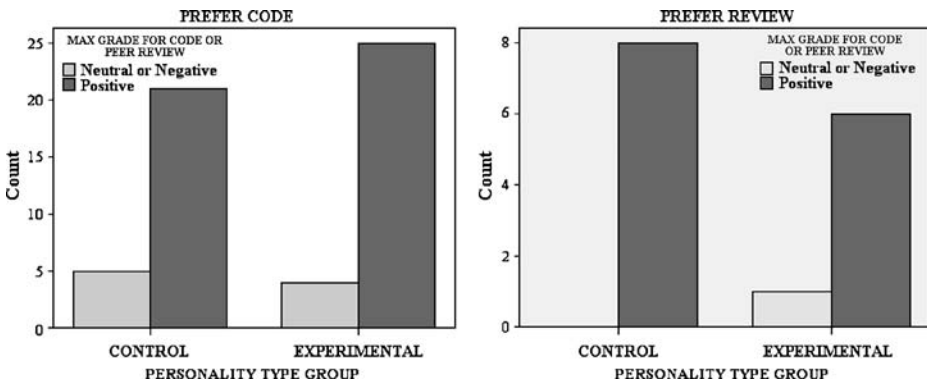
Fig. 7 Communication satisfaction ratio

Fig. 8 Degree of nuisance


observing positive attitude (4 and 5 in the original ordinal measurement scales), we note better collaboration satisfaction (80.6% vs. 50%), better knowledge acquisition (52.8% vs. 26.5%) and better participation with respect to communication satisfaction (69.4% vs. 41.2%). On the other hand, the attitudes of the experimental group against the control with respect to nuisance disturbances (16.7% vs. 26.5%) and coding or reviewing preferences (86.2% vs. 80.8% and 85.8% vs. 100% respectively) were not significantly different. Coding preference (driver preference) was rated high from both groups, while nuisance disturbances were rated low from both groups.

6 Interpretation of the Statistical Results

From all the statistical analyses, the results show that the two groups exhibit important differences in pair performance and pair collaboration-viability. The differences suffice to


Fig. 9 Driver or navigator preference

predict the personality type of a pair from certain performance metrics, as shown from the results of DA. Based on the aforementioned results, the stated hypotheses can now be answered.

6.1 Hypotheses Testing

H01: *Heterogeneous developer personalities and temperaments do not affect pair performance.*

The statistical tests for all performance variables (separately for both tasks and for the totals of the variables) indicated significantly better results for the heterogeneous types. The group of pairs with heterogeneous types had much more communication transactions than the control group. Correlation, concerning communication, is strongly positive for the experimental group and almost non-existent for the control group and this can be thought as providing interesting insight. Moreover the group of pairs with heterogeneous types produced better results (better points and better acceptance tests) than the control group. Consequently, the null hypotheses H01 is not valid and must be rejected in favor of the alternative one.

H02: *Heterogeneous developer personalities and temperaments do not affect pair collaboration-viability.*

The preliminary results from the data analysis of the questionnaires also revealed better pair collaboration-viability for the group with heterogeneous types. Representing higher scores (4 and 5 in the measurement scales) as a positive attitude we noted better collaboration satisfaction, better knowledge acquisition and better participation, thus H02-hypothesis must be rejected in favor of the alternative one.

6.2 Validity Evaluation

Validity is divided into four types, each of which describes one type of threat to the validity of the results as well as a specific methodological approach to address it (Campbell and Stanley 1996; Cook and Campbell 1979; Judd et al. 1991). The four validity types and their associated threats are strongly interrelated (Trochim 1999; Wohlin et al. 2000). In this section the four validity threats are discussed, as well as the approaches we took to address them.

6.2.1 Internal Validity

The internal validity threats concern different influences which can affect the independent variable with respect to causality, which means that they can affect the conclusion about a possible causal relationship between treatment and outcome. In a multiple groups experiment, as ours (two groups), the threats are due to the different behavior of the different groups involved in the experiment and comprise a category named “interactions with the selection”. These threats, according to (Cook and Campbell 1979) are:

- *History*: The pairs were formed and allocated to both groups half an hour before the start of the experiments so there was no time before or during the experiment in which the subjects of the two groups could collaborate.
- *Maturation*: Fatigue effects, loss of motivation, or learning effect are some factors that can cause maturation effects. The experiment was prepared and executed in

separate sessions, with a week interval between each session. The execution time was about three hours, so fatigue was not important. Moreover, exactly the same body of information and training was given to both groups during the preparation sessions.

- *Instrumentation*: Differences in the way of measuring can cause this type of threats. We used the same evaluation system for both assignments and groups. We emphasize the fact that correctness points have been assigned by mere consultation of a predefined checklist to ensure objectiveness. However, two threats concerning instrumentation must be discussed here, as well as the followed approaches to confront them.
 1. Communication transactions recording: We were aware that the manual data collection would need enhanced responsibility from navigators and would produce considerable overhead and continuous “context switch” between reviewing and recording. To encourage students’ involvement and improve their willingness and contribution in the measurement activity we trained them in the recording process. During the training we emphasised that measurement concerns the software process and not their personal performance. During the lecture sections and the lab sessions we thoroughly taught them how to classify the communication transactions in pair programming and how to fill the forms during the experiment. Care was taken to reduce the overhead of metrics collection and streamline the recording process by using already printed forms with predefined columns. Before the execution of the main experiment, the entire recording process was tested in the pilot run of the experiment procedure. The teaching staff’s role was to monitor the recording process, answering transactions-related questions and handling possible problems.
 2. KTS-test limitations: A limitation of the KTS-test, as with other self-reporting psychological testing instruments, is that it requires honesty on the part of the respondent. If a person gives false answers in order to present a particular image, then the test will not provide an accurate indication of his true preferences. This effect cannot be eliminated, only minimised. We tried to minimise this threat by guaranteeing students anonymity, assuring them that data would be used only for pair formation process. Moreover, we informed them about the type of questions they had to answer in the training session. Another limitation of the MBTI/KTS tests concerns their statistical structure. Two people are placed in the same category (i.e. ‘E’) even if one has a moderate ‘E’ score and one has an extreme ‘E’ score. This happens when a person is close to the center of the scale. In our study, facing this problem with two students, we allocated them in different groups of pairs, trying to balance possible undesirable biases.
- *Selection*
 Subject selection affects performance and knowledge handling. We addressed this issue very carefully.
 - ✓*Randomization*: The subjects (70 students) were volunteers, formed pairs randomly and according to their personality and temperament inventories, and were randomly assigned to the control and experiment groups.
 - ✓*Knowledge*: All subjects had about the same level of knowledge.

✓*Differences among subjects*: All students appeared to have roughly the same expertise and similar skills. However, it is known that performance and productivity can vary greatly among programmers, which might present a potential threat to our experiment.

✓*Subject motivation*: Students were motivated by the educational benefits they would gain from their participation and by practicing their skills in a more controlled setting. They were also motivated by a bonus (10%) grade in their final assessment grade only after passing their final assessment exams.

✓*Persistence effects*: It was the first time the students participated in an experiment, so there is no such threat.

- *Time precision*: Time was precisely specified for the assignment completion.
- *Mortality*: This effect is caused by subjects dropping out of the experiment. We have not faced such an effect.
- *Social threats*

✓*Diffusion or imitation of treatments*: The best guarantee against this threat was subjects' total ignorance of the experimental hypotheses and objectives. During experiment execution we took all the measures needed to avoid unfair collaboration between subjects.

✓*Compensatory rivalry-Resentful demoralization*: These threats have to do with the competition between the groups, meaning (the former) that subjects try to perform well and (the latter) the opposite one, i.e. subjects may give up and not perform well. In the experiment the mechanism for team composition was not known to the subjects, therefore subjects were not aware of the experiment assumptions.

In general, the participants were under full control of the researchers during the experiment. Concentrating on developer personalities/temperaments and their impact on communication and collaboration-viability, we tried to eliminate possible threats. We scattered Extroverts and Introverts equally within the two groups, knowing that these two different types potentially influence communication. We let other factors such as gender and race to vary randomly, balancing possible effects. The average skill and knowledge level of the pairs was also balanced, but there may have still been unobserved skill or knowledge differences affecting the performance of the pairs.

6.2.2 External Validity

There are pros and cons to student experiments. A limitation is the complete applicability of results with students to the professional environment. By making the experimental environment as realistic as possible, the threats to external validity are reduced and research results can be generalised to an extent to the population under study and to other research settings. There are three types of interactions with the treatment: people, place and time.

- *Interaction of selection and treatment*

We consider that the sample of subjects (70 students in the main experiment and 15 students in the pilot run of the experiment procedure) in our study was quite representative of the total population of students, even though its size hindered us from having pairs with all possible combinations of personalities and temperaments. A much

larger sample would probably allow us to form pairs with all possible type combinations.

- *Interaction of history and treatment*

The experiment was conducted and executed during normal semester days and hours.

No historical events at the time of the study happened and no other casual relationships were observed.

- *Interaction of setting and treatment*

This threat has to do with the lack of correct experimental setting or the lack of representative material. The small tasks we used in the experiment and the short time study may limit generalizability.

Generally:

- An important limitation is that pair effectiveness was studied over short time. Results may be different for long term projects and collaborations.
- Switching of roles (driver-navigator) in an hour time interval may be not fully representative of pair programming applied in practice.
- Our study avoided noise disturbances and interruptions, but in reality things may be different.
- Information about whether the pair members were familiar with each other before the experiment was not collected.

6.2.3 Construct Validity

Construct validity describes how the obtained theoretical construction knowledge relates to the stated hypotheses. The main threats to construct validity are two: design and social threats (Cook and Campbell 1979; Wohlin et al. 2000).

- *Design threats*

These threats concern the design of the experiment.

✓*Inadequate preoperational explication of constructs*: We created our measurement system using the same constructs and measures as described in the literature for team and group construction (see Sections 4.4 and 4.5).

✓*Mono-operation bias*: These threats pertain to a single independent variable, single cause, program or treatment in an experiment. To minimize this threat we assigned two different tasks, while the assessment of the two assignments for each pair was multi-sided, including many different assessment methods such as transactions forms, correctness points, passed acceptance tests and questionnaires.

✓*Mono-method bias*: This threat concerns the use of a single type of measures. In our experiment we used different types of measures for different constructs. However, for practical reasons we did not use different ways of measurement for each construct. Another threat concerning collaboration-viability is the use of one question per category in the questionnaire.

✓*Confounding constructs and levels of constructs*: The subjects in our experiments had the same level of knowledge and were assigned randomly to pairs and groups according to their personality and temperament inventories. A problem was raised by the small percentages in the experiment sample of two temperament types (10% for

artisans-SP and 8.6% for rationals-NT). This fact did not permit all possible temperament types' combinations in the experiment. To address this issue we had to form pairs of only two temperament combinations in the control group and five combinations in the experimental group. An experiment with a larger number of students (e.g. with 500 students) would probably allow all possible combinations). Another potential problem could be the absence of pair jelling prior to the execution of the experiment.

✓*Interaction of different treatments*: When subjects are involved in many studies, then treatments may interact. The subjects we used were involved only in the experiments described here.

✓*Interaction of testing and treatment*: This threat can be caused when the testing or measurement itself make the subjects sensitive or receptive to the treatment. Only communication measurement could cause such a threat. But the measurement of communication was embedded in the pair programming activity, reducing the risk of making the groups more sensitive or receptive to the treatment.

✓*Restricted generalizability across constructs*: In some experiments the treatment affects the studied construct positively or negatively, making the results hard to generalize. We have not faced any impact on the used constructs, nor any drastic negative consequence or side effect of the treatment.

- *Social threats*

These threats concern the behavior of the subjects and the experimenters.

✓*Hypothesis guessing*: Subjects might only guess that their personality could be involved in some way in the experiment. However, subjects were neither aware of what we intended to investigate (combination of pair temperaments), were not aware of their own temperament, nor were they informed about the actual hypothesis examined.

✓*Evaluation apprehension*: The students involved in our study were volunteers, so they were not afraid of being evaluated.

✓*Experimenter expectancies*: This threat can be caused when the researcher bias the results of a study, either consciously or unconsciously. Our study aims at comparing pairs with heterogeneous personalities and temperaments and pairs with homogeneous personalities and temperaments, in terms of effectiveness, without having any specific preference or expectation from the experiment.

6.2.4 Conclusion Validity

Conclusion validity describes the statistical validity of the results.

- *Reliability of measures*: In addition to the already known measures defined in literature and used in existing empirical studies on team and group construction, we considered the communication and the collaboration-viability variables. The communication variable was used to capture scheduled communication transactions in relation to time. As we already mentioned, a restriction concerning communication metrics, namely the bidirectional limitation in the communication modes, resulted from the design of the experiments. Nevertheless, the multiple assessment methods used and the objectivity in the assessment process ensured the reliability of the measurements.

- *Reliability of treatment implementation*: This threat can be caused when the treatment implementation is not standard for different subjects. In our experiment the treatment implementation was similar between different subjects, pairs and group of pairs.
- *Violated assumption of statistical tests*: Some statistical tests are more robust to violated assumptions than others and violating the assumptions may lead to wrong conclusions. We used many different statistical methods and tests to avoid this kind of threat.
- *Fishing and the error rate problem*: This is a threat occurring when a researcher is “fishing” for a specific result when he or she applies repeatedly statistical tests on the same data, under slightly different assumptions or conditions, assuming that all these tests are independent of each other. In such a case, there is an increased probability to find statistically significant differences in the data when such differences do not really exist in population. In our analysis there is no such threat, since we used only the appropriate tests for different hypotheses.
- *Random irrelevancies in experimental setting*: Disturbances by noises or sudden interruptions can cause problems in the execution of the experiments. We did not face such disturbances.
- *Random heterogeneity of subjects*: This threat states that the selected groups must not be very heterogeneous or very homogeneous. The sample used can be considered homogeneous, but on the other hand the random separation of the groups assured that the differences in the outcome were due to the different treatments.
- *Low statistical power*: Low statistical power in tests can lead to false conclusions, by rejecting a true hypothesis or not rejecting a false one. In our experiments we tried to eliminate this risk in the design phase using two large enough groups (in a small scale-study) and choosing the most appropriate statistical tests, as theory and practice suggest.

In summary, most threats were under control. There are researchers who suggest that running experiments with students reduces the validity of the study (Kitchenham et al. 2002). We believe that running experiments with students offers some advantages that can not be found in an industrial environment. The controlled experiment described in this study would be hard to conduct in a fully industrial environment, considering time and cost. We needed about one month to prepare and execute the experiment, in which 70 subjects in 35 pairs participated. On the other hand, it should be noted that experiment concerned a short-time study on pair effectiveness.

7 Conclusions and Future Work

This experiment studies the impact of developer personalities and temperaments on pair effectiveness described in terms of pair performance and pair collaboration-viability. The experiment was executed in two phases. In the first phase we established students' personality and temperament inventories, using them in pair formation and pair allocation process. In the second phase we executed the experiment comparing pairs with heterogeneous personalities and temperaments with pairs of homogeneous types, in various terms of effectiveness. The results have shown better performance and collaboration-viability for pairs with heterogeneous personalities and temperaments. The partial analysis of the data for the separate performance variables, using different

statistical tests, showed important advantages for the heterogeneous personality/temperament pairs in terms of communication, design and code correctness and velocity (less time to complete the tasks).

The most interesting results were obtained from the communication variable which was included among the performance variables to capture communication in real activities occurring during pair programming, where each developer's contribution is difficult to establish. The data analysis of this variable has shown that design and code correctness for pairs with heterogeneous types is positively correlated with communication transactions, a fact that was not observed for pairs of the same types. It appears that more communication leads to higher correctness, a finding that is in line with Forsyth's and Cockburn's findings, providing partial support to Cockburn's claim on the positive impact of heterogeneous personalities on the minimization of the communication gap.

The preliminary results from the data analysis of the questionnaires also showed better collaboration and viability for the group with heterogeneous types. More specifically the participants' satisfaction regarding collaboration, knowledge acquisition and participation was significantly higher for the heterogeneous pairs.

We believe that the most important consequences of these results can be summarised as follows:

- The identification, understanding and combination of developer variations can improve pair communication and pair collaboration-viability.
- By monitoring the effectiveness of pair formations and rotations (in regular mini retrospectives—communication and collaboration reviews), developers can learn about themselves and how they will effectively communicate and collaborate.

The contribution of this preliminary empirical study is to provide some evidence that may assist the pair forming process. However, further empirical studies must be conducted in order to investigate causes for the difference between heterogeneous and homogeneous teams of pairs (e.g. qualitative studies with interviews), continuing this endeavor. The limitations of the study concern the KTS-test, the use of students as subjects, the absence of pair jelling and the assessment of pair effectiveness over short time. We plan to continue our experimental work testing the impact of developer personality and temperament types on pair performance and pair effectiveness in long-time studies. More specifically, we plan to investigate extended-time pair effectiveness, testing the impact of all types' combinations (MBTI-personality types and KTS-temperaments types) on pair communication and pair collaboration-viability in short and long-time periods.

Acknowledgment We would like to express our heartfelt gratitude to psychologist Dr. Anastasia Psalti, of the Department of Medicine and Health Care, at the Alexander Technological Educational Institution, Thessaloniki, Greece, for the help she provided in all phases of the experiment and the pilot run of the experiment procedure. We would like to thank the students for their participation in the experiment and both Departments of Informatics of the Alexander Technological Educational Institution and of the Aristotle University, for contribution and support in the completion of the experiment.

Appendix 1

Communication Transactions Form

COMMUNICATION TRANSACTIONS FORM

Dev. -1 Name:

Dev. -2 Name:

Dev. -1 Code Nbr:

Dev. -2 Code Nbr:

Start time:

Finish time:

A/A	Dev. Code Nbr.	Requirements (add /delete /modify)	Specification/Design (add /delete /modify)	Coding (add /delete /modify)	Unit testing (add /delete /modify)	Review code/ design	Other (describe)
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12	:	:	:	:	:	:	:

(Write Developer Code Number and tick the appropriate communication mode)

A/A	Dev. code No.	Requirements (add/delete / modify)	Specification/ Design (add/delete / modify)	Coding (add/delete / modify)	Unit testing (add /delete/ modify)	Review code/ design	Other (describe)
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12	:	:	:	:	:	:	:

Write developer code number and tick the appropriate communication mode

Appendix 2

Questionnaire (for pair collaboration-viability)

Dev. Name:

Dev. Code Nbr:

1 - Using the ordinal scale 1 (min) – to - 5 (max), please rate your **collaboration satisfaction**:

1 2 3 4 5

----------------------	----------------------	----------------------	----------------------	----------------------

2 - Using the ordinal scale 1 (min) – to - 5 (max), please rate your **knowledge acquisition**:

1 2 3 4 5

----------------------	----------------------	----------------------	----------------------	----------------------

3 - Using the ordinal scale 1 (min) – to - 5 (max), please rate your **communication satisfaction**:

1 2 3 4 5

----------------------	----------------------	----------------------	----------------------	----------------------

4 - Using the ordinal scale 1 (min) – to - 5 (max), please rate the **nuisance**:

1 2 3 4 5

----------------------	----------------------	----------------------	----------------------	----------------------

5 - Using the ordinal scale 1 (min) – to - 5 (max), please rate your **driver or navigator preference** (*Please check only one of the two choices*):

1 2 3 4 5

Driver

----------------------	----------------------	----------------------	----------------------	----------------------

Navigator

----------------------	----------------------	----------------------	----------------------	----------------------

6 – Do you prefer **voluntary or mandatory pair formation**? (*Please check only one of the two choices*)

Voluntary ☐

Mandatory ☐

Remarks (*Please note your comments*)

Appendix 3

The following descriptions refer to the Keirsey type, each type contribution to the population, and descriptive list of identifying characteristics. These descriptions have been extracted from Keirsey and Bates (1984).

Personalities

Extroverting (E)–Introverting (I)

Extroverts get their energy from the outside world, from experiences and interactions, while Introverts from within themselves, from their internal thoughts, feelings, and reflections. Extroverts tend to talk easily about anything, whereas Introverts are comfortable with long silences. Introverts prefer finished ideas, namely to read and think about something before start talking, so we must give them time to process new information, especially in meetings. For interactions with users and management, Extroverts are better in talking (and getting responses), and at presenting ideas than Introverts.

Sensing (S)–Intuiting (N)

Sensors gather information linearly and tend to take things literally and sequentially, preferring tangible results clearly described. They are observant of what is happening around them, and are especially good at recognizing the practical realities of a situation. Their concentration to details makes them the most capable programmers. Intuitives like to gather information more abstractly (seeing the big picture), focusing on the relationships and connections between facts. They prefer to speak in concepts and global futuristic ideas and they are good at seeing new possibilities and different ways of doing things. Therefore, they can be used more as system analysts determining users/clients' needs and identifying problems in systems.

Thinking (T)–Feeling (F)

Thinkers make decisions based on objective information, while feelers are based on subjective information. Thinkers, tend to be logical, critical, orderly, and prefer to work with facts. They examine carefully cause and effect of a choice or action to enhance their problem-solving abilities. Their tendency to be logical is needed mostly in programming. Feelers, driven by personal values, make decisions considering what is important to themselves and others. Their strengths include understanding, appreciation and support of others. They tend to be more people-oriented and good team-builders.

Judging (J)–Perceiving (P)

The most problems concerning communication are caused by differences in the work styles. Judgers tend to live in an orderly and planned way, with detailed schedules. They like things decided and concluded and they prefer a bad decision than no decision. Perceivers tend to live in a flexible, spontaneous way, are based on experience and have open issues. They like to explore every possibility, and consequently have difficulty making decisions working on deadlines. On the contrary Judgers prefer to avoid last-minute stresses, and try to conclude quickly: once they have found a good solution they accept it and move on to something else. A combination of Judgers and Perceivers in a pair would help to ensure that the best solution will be found in a reasonable time.

Temperaments

Artisans and guardians prefer concrete communications, while idealist and rational prefer more abstract communications. Artisan and rational types prefer a cooperative path to goal accomplishment, while guardian and idealist types prefer more a utilitarian approach. Artisans, combining a sensor's sense of reality and a perceiver's spontaneity, feel most comfortable with clear and direct communications in cooperative pursuit of a goal. Possessing a correct sense of timing, they are great start-up persons, are lateral thinkers, problem solvers and great brainstormers. They are troubleshooters proposing practical

solutions and making quick decisions. Their adaptability and sense of innovation can help the pair achieve requirements with new technology.

Guardians, combining a sensor's sense of reality and a judger's desire for reaching a conclusion, are traditionalists and stabilizers. They are good as administrators, establishing policies, rules, schedules, regulations and a functioning hierarchy. However, they do not easily perceive project delays or problems. Because of a strong need for conclusions, they may not allow enough time for processing. They tend to want things to remain the same, without allowing for sparks of creativity or innovation. They will not be very successful in times of rapid change.

Combining an intuitive's focus on the relationships and a feeler's preference in personal values, Idealists are successful in leveraging their own and other people skills. They like to guide others and are committed to the progress of the people in a given setting. They are also transactional as Artisans, excellent communicators and people motivators. An Idealist developer adds a personalised touch to a pair, contributing to team spirit and morale. However, he does not react well to sudden changes.

Rationalists, combining the objectivity of the thinkers and the visionary of the intuitives, are described as theorists, innovators, systems planners and architects of change. They focus on competence and excellence, valuing high logic and reason, adopting a logical and strategic analysis approach of resolving issues. Thus their most practiced and successful operations tend to be planning, inventing and configuring.

Appendix 4

Samples of checklists and the associated grading schemes.

Coffee Machine

The coffee machine serves coffee for 35 cents, with or without sugar and creamer. The coffee list includes: black (without sugar and cream powder), white (with cream powder), sweet (with sugar), sweet_white (with sugar and cream powder). The front panel will have four buttons (choice menu with the four types of coffee). There will also be a coin return button. The machine will take euro coins only (max 1 Euro), and give changes. This design consists of 12 classes:

Classes	Responsibility-driven design (RD)
CoffeeMachine	Initiates the machine; knows how the machine is put together; handles input
CashBox	(to be implemented)
FrontPanel	Knows selection; coordinates payment; delegates drink making to the product.
Product	Knows its recipe and price.
ProductRegister	Knows what products are available.
Recipe	Knows the ingredients of a given product; tells dispensers to dispense ingredients in sequence.
Dispensers	Knows which ingredient it contains; controls dispensing; tracks amount it has left.
DispenserRegister	Knows what dispensers are available
Ingredient	Knows its name only.
Output	Knows how to display text to the user.
Input	Knows how to receive command-line input from the user
Main	Initializes the program

First Assignment—Cash Administration

<i>Please make the following changes:</i>	Sub Tasks	Points
Start the machine, inputting a cash amount (i.e. 10,000 cents).	Design	0.5
	Code-unit-tests	1.0
Make customer transactions:	Design	0.5
Give change, checking the amount.	Code-Unit-tests	0.5
Display the proper message if the amount is not enough, and	->>-	0.5
Input new amount.	->>-	0.5
End of function:	Design	0.5
Print the total amount of transactions and	Code-unit-tests	0.5
The cash balance.	->>-	0.5

Second Assignment—Transactions' Statistics

<i>Please make the following changes:</i>	Sub Tasks	Points
<i>Make customer transactions:</i>	Design	0.5
Log each sale transaction.	Code-unit-tests	0.5
<i>End of function:</i>	Design	0.5
1. Sort transactions in descending order.	Code-Unit-tests	2.0
<i>End of function:</i>	Design	0.5
2. Print in a descending order the most sold coffee types.	Code-unit-tests	1.5

The final grade is calculated by summing the points of each sub task.

Personal Software Process and PSP are registered service marks of Carnegie Mellon University.

References

- Ahmad K, Hammond N (2004) Taxonomy of analysis levels of learning effectiveness in collaborative concept mapping. Proceedings of the First International Conference on Concept Mapping, Pamplona, Spain
- Arisholm E, Gallis H, Dyba T, Sjöberg D (2007) Evaluating pair programming with respect to system complexity and programmer expertise. *IEEE Trans Softw Eng* 33(2):65–86 doi:[10.1109/TSE.2007.17](https://doi.org/10.1109/TSE.2007.17)
- Basili V, Rombach H (1988) The TAME project: towards improvement-oriented software environments. *IEEE Trans Softw Eng* XIV(6):758–773 doi:[10.1109/32.6156](https://doi.org/10.1109/32.6156)
- Basili V, Weiss D (1984) A methodology for collecting valid software engineering data. *IEEE Trans Softw Eng* SE-10:728–738
- Basili V, Shull F, Lanubille F (1999) Building knowledge through families of experiments. *IEEE Trans Softw Eng* 25(4):456–473 doi:[10.1109/32.799939](https://doi.org/10.1109/32.799939)
- Beck K (2000) Extreme programming explained: embrace change. Addison-Wesley, Boston, USA
- Berens LV (1998) Understanding yourself and others: an introduction to temperament. Telos, Huntington Beach, CA
- Bradley JH, Hebert FJ (1997) The effect of personality type on team performance. *J Manage Dev* 16(5):337–353 doi:[10.1108/02621719710174525](https://doi.org/10.1108/02621719710174525)
- Briand L, Bunse C, Daly J, Differding C (1997) An experimental comparison of the maintainability of object-oriented and structured design documents. *Empir Softw Eng* 2(3):291–312 doi:[10.1023/A:1009720117601](https://doi.org/10.1023/A:1009720117601)
- Briand L, Arisholm E, Counsell F, Houdek F, Thevenod-Fosse P (1999) Empirical studies of object-oriented artifacts, methods, and processes: state of the art and future directions. *Empir Softw Eng* 4(4):387–404 (18)

- Campbell DT, Stanley JC (1996) Experimental and quasi-experimental designs for research. Rand McNally, Chicago, pp 1–12, 34–71
- Canfora G, Cimitile A, Visaggio CA (2004) Working in pairs as a means for design knowledge building: an empirical study. 12th IEEE International Workshop on Program Comprehension, pp 62–68. ISBN: 0-7695-2149-5
- Captex LF (2003) Personality types in software engineering. *Int J Hum Comput Stud* 58(2):207–214 doi:10.1016/S1071-5819(02)00137-4
- Center for Applications of Psychological Type (2006) MBTI Code of Ethics. http://www.capt.org/The_MBTI_Instrument/Ethical_Use.cfm. Last visited: 26 January 2006
- Chao J, Atli G (2006) Critical personality traits in successful pair programming. Proceedings of AGILE 2006 Conference (AGILE'06), pp 89–93
- Chong J, Hurlbutt T (2007) The social dynamics of pair programming. 29th International Conference on Software Engineering (ICSE'07), pp 354–363
- Cockburn A (1998, May/June) The coffee machine design problem: part 1 & 2. *C/C++ user's journal*
- Cockburn A (2002) Agile software development. Addison-Wesley, Boston, USA
- Cockburn A, Williams L (2000) The costs and benefits of pair programming. 1st International Conference on Extreme Programming and Flexible Processes in Software Engineering, Italy (XP2000)
- Coleman JS (1990) The foundations of social theory. Harvard University Press, Cambridge, MA
- Cook TD, Campbell DT (1979) Quasi-experimentation: design and analysis for field settings. Rand McNally, Chicago, IL, USA
- Curtis B, Krasner H, Iscoe N (1988) A field study of the software design process for large systems. *Commun ACM* 31:1268–1287 doi:10.1145/50087.50089
- Dutoit A, Bruegge B (1998) Communication metrics for software development. *IEEE Trans Softw Eng* 24(8):615–628
- Fenton N (1991) Software metrics, a rigorous approach. Chapman & Hall, London
- Ferdinand P (1998) Facilitating communication. *IEEE Softw* 15(5):92–96 doi:10.1109/52.714835
- Forsyth DR (2006) Group dynamics, 4th edn. Belmont, CA, USA
- Gallis H, Arisholm E, Dybå T (2002) A transition from partner programming to pair programming—an industrial case study. Pair Programming Work Shop in 17th Annual ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'02), pp 1–8
- Gallis H, Arisholm E, Dybå T (2003) An initial framework for research on pair programming. Proceedings of the 2003 International Symposium on Empirical Software Engineering (ISESE'03), p 132
- Gorla N, Lam YW (2004) Who should work with whom? Building effective software project teams. *Commun ACM* 47(6):79–82 doi:10.1145/990680.990684
- Halfhill TR, Sundstrom E, Lahner J, Calderone W, Nielsen TM (2005) Group personality composition and group effectiveness: an integrative review of empirical research. *Small Group Res* 36(1):83–105 doi:10.1177/1046496404268538
- Highsmith J (2002) Agile software development ecosystems. Addison-Wesley, Boston, MA, USA
- Hulkko H, Abrahamsson P (2005, May) A multiple case study on the impact of pair programming on product quality. ICSE05. St. Louis, Missouri, USA, pp 495–504
- Humphrey WS (1996) Using a defined and measured personal software process. *IEEE Softw* 13:77–88 doi:10.1109/52.493023
- Humphrey WS (1997) Managing technical people: innovation, teamwork, and the software process. Addison Wesley, Reading, MA
- Jensen RW (2003) A pair programming experience. *CrossTalk. J Def Softw Eng* 16:22–24
- Johnson PM, Kou H, Agustin JM, Chan C, Moore CA, Miglani J et al (2003) Beyond the personal software process: metrics collection and analysis for the differently disciplined. International Conference on Software Engineering. Portland, Oregon. Proc 2003, pp 641–646
- Judd CM, Smith ER, Kidder LH (1991) Research methods in social relations, 6th edn. Holt, Rinehart and Winston, Inc., Toronto
- Katira N, Williams L, Wiebe E, Miller C, Balik S, Gehringer E (2004) On understanding compatibility of student pair programmers. ACM Technical Symposium on Computer Science Education, SIGCSE04, pp 7–11
- Katira N, Williams L, Osborne J (2005) Towards increasing the compatibility of student pair programmers. International Conference on Software Engineering-ICSE05. St. Louis, Missouri, USA, pp 625–626
- Keefer G (2005) Pair programming: an alternative to reviews and inspections? *Cutter IT Journal*, AVOCA
- Keirse D (1998) Please understand me II. Prometheus Nemesis Book Company, Del Mar, CA
- Keirse D, Bates M (1984) Please understand me.. Prometheus Nemesis Books, Del Mar, California
- Khamesan A, Hammond N (2004) Synchronous collaborative concept mapping via ICT: learning effectiveness and personal and interpersonal awareness. Proceeding of 1st International Conference on Concept Mapping (1st CMC, 2004), Spain

- Kirkman BL, Tesluk PE, Rosen B (2001) Assessing the incremental validity of team consensus ratings over aggregation of individual-level data in predicting team effectiveness. *Person Psychol* 54:645–667 doi:[10.1111/j.1744-6570.2001.tb00226.x](https://doi.org/10.1111/j.1744-6570.2001.tb00226.x)
- Kitchenham B, Pfleeger S, Pickard L, Jones P, Hoaglin D, El Emam K et al (2002) Preliminary guidelines for empirical research in software engineering. *IEEE Trans Softw Eng* 28(8):721–734 doi:[10.1109/TSE.2002.1027796](https://doi.org/10.1109/TSE.2002.1027796)
- Langley A (1999) Strategies for theorizing from process data. *Acad Manage Rev* 24(4):691–711 doi:[10.2307/259349](https://doi.org/10.2307/259349)
- Laplante P, Neil C (2006) Antipatterns. Identification, refactoring, and management. Auerbach, USA
- Leana CR, Van Buren HJ (1999) rganizational social capital and employment practices. *Acad Manage Rev* 24(3):538 doi:[10.2307/259141](https://doi.org/10.2307/259141)
- Lui KM, Chan KCC (2003) When does a pair outperform two individuals? Fourth international conference in Extreme Programming and Agile Processes in Software Engineering (XP2003). LNCS 2003. Springer, pp 225–233
- Lui KM, Chan KCC (2006) Pair programming productivity: novice–novice vs. expert–expert. *Int J Human-Comput Stud* 64(9):915–925
- Macias F (2004) Empirical assessment of extreme programming. PhD thesis, University of Sheffield (August)
- McDonald S, Edwards H (2007) Who should test whom. *CACM* 50(1):66–71
- McDowell C, Werner L, Bullock H, Fernald J (2002) The effect of pair programming on performance in an introductory programming course. *ACM Special Interest Group of Computer Science Educators*, Covington, KY, pp 38–42
- Melnik G, Maurer F (2002) Perceptions of Agile practices: a student survey. *XP/Agile Universe*, Chicago, IL, pp 241–250
- Melnik G, Maurer F (June 2004) Direct verbal communication as a catalyst of Agile knowledge sharing. *Proceedings of the Agile Development Conference*. IEEE Computer Society, pp 21–31
- Müller M (2003) Are reviews an alternative to pair programming? 7th International Conference on Empirical Assessment in Software Engineering, UK, volume 9(4), pp 335–351
- Müller M (2005) Two controlled experiments concerning the comparison of pair programming to peer review. *J Syst Softw* 78(2):166–179 doi:[10.1016/j.jss.2004.12.019](https://doi.org/10.1016/j.jss.2004.12.019)
- Müller M, Tichy W (2001) Case study: extreme programming in a university environment, *International Conference on Software Engineering (ICSE)*, pp 537–544
- Myers I (1962) *Manual: the Myers–Briggs type indicator*. Consulting Psychologists Press, Palo Alto, California
- Nawrocki J, Wojciechowski A (2001, April) Experimental evaluation of pair programming. 12th European Software Control and Metrics Conference (ESCOM 2001), London, UK
- Nielsen TM, Sundstrom E, Halfhill TR (2005) Group dynamics and effectiveness: five years of applied research. In: Wheelan SA (ed) *The handbook of group research and practice*. Sage, Thousand Oaks, CA, pp 285–311
- Nosek J (1998) The case for collaborative programming. *Commun ACM* 41(3):105–108 doi:[10.1145/272287.272333](https://doi.org/10.1145/272287.272333)
- Ohira M, Yokomori R, Sakai M, Matsumoto K, Inoue K, Torii K (2004) Empirical project monitor: a tool for mining multiple project data. 1st International Workshop on Mining Software Repositories, Tokyo, Japan, pp 141–150
- Saeki M (1995) Communication, collaboration, and cooperation in software development—how should we support group work in software development? *Asia-Pacific Software Engineering Conference*. Brisbane, Australia, pp 12–20
- Seaman C, Basili V (1997) An empirical study of communication in code inspections. 19th International Conference on Software Engineering, Boston, pp 96–106 (May)
- Sfetsos P, Angelis L, Stamelos I (2006) Investigating the extreme programming system—an empirical study. *Empir Softw Eng* 11(2):269–301
- Sillitti A, Janes A, Succi G, Vernazza T (2003) Collecting, integrating and analyzing software metrics and personal software process data. *Proc EUROMICRO* 2003:336–342
- Stevens T (1998) The effects of roles and personality characteristics on software development team effectiveness. Dissertation for Doctor of Philosophy, Virginia Polytechnic Institute and State University
- Stoyanova N, Kommers P (2002) Concept mapping as a medium of shared cognition in computer supported collaborative problem solving. *J Interact Learn Res* 13(1/2):111–133
- Sundstrom E, De Meuse K, Futrell D (1990) Work teams. *Am Psychol* 45:120–133 doi:[10.1037/0003-066X.45.2.120](https://doi.org/10.1037/0003-066X.45.2.120)
- Thomas L, Ratcliffe M, Robertson A (2003, February) Code warriors and code-a-phobes: a study in attitude and pair programming, *Proceedings of the 34th SIGCSE technical symposium on Computer science education*, Reno, Nevada, USA, pp 363–367

- Tomayko JE (2002) A comparison of pair programming to inspections for software defect reduction. *J Comput Sci Educ* 12:213–222 doi:[10.1076/csed.12.3.213.8614](https://doi.org/10.1076/csed.12.3.213.8614)
- Trochim W (1999) The research methods knowledge base, 2nd edn. Cornell Custom Publication, Cornell University, Ithaca, NY
- Williams L (2001) Integrating pair programming into a software development process. 14th Conference on Software Engineering Education and Training, USA, vol 2001, pp 27–36
- Williams L, Kessler R (2003) Pair programming illuminated. Addison-Wesley, USA, Boston
- Williams L, Kessler R, Cunningham W, Jeffries R (2000) Strengthening the case for pair-programming. *IEEE Softw* 17:19–25 doi:[10.1109/52.854064](https://doi.org/10.1109/52.854064)
- Williams L, Shukla A, Antón AI (2004) An initial exploration of the relationship between pair programming and Brooks' law. Agile Development Conference, Salt Lake City, Utah, USA, volume issue, 22–26 June 2004, pp 11–20
- Williams L, Layman L, Osborne J, Katira N (2006) Examining the compatibility of student pair programmers. *Proceedings of Agile 2006-AGILE06*, pp 411–420
- Wohlin C, Runeson P, Höst M, Ohlson M, Regnell B, Wesslén A (2000) Experimentation in software engineering: an introduction. Kluwer, USA



Panagiotis Sfetsois is an Assistant Professor at the Department of Informatics at the Alexander Technological Educational Institute of Thessaloniki, Greece. He received his B.Sc. in Computer Science and Statistics from the University of Uppsala, Sweden (1981), and the Ph.D. degree in Computer Science from the Aristotle University of Thessaloniki (2007). His Ph.D. Thesis was on “Experimentation in Object Oriented Technology and Agile Methods”. His research interests include empirical software evaluation, measurement, testing, quality, agile methods and especially extreme programming.



Ioannis G. Stamelos is an Associate Professor of Computer Science at the Aristotle University of Thessaloniki, Dept. of Informatics. He received a degree in Electrical Engineering from the Polytechnic School of Thessaloniki (1983) and the Ph. D. degree in Computer Science from the Aristotle University of Thessaloniki (1988). He teaches object-oriented programming, software engineering, software project management and enterprise information systems at the graduate and postgraduate level. His research interests include empirical software evaluation and management, software education and open source software engineering. He is author of 90 scientific papers and member of the IEEE Computer Society.



Lefteris Angelis received his B.Sc. and Ph.D. degree in Mathematics from Aristotle University of Thessaloniki (A.U.Th.). He works currently as an Assistant Professor at the Department of Informatics of A.U.Th. His research interests involve statistical methods with applications in software engineering and information systems, computational methods in mathematics and statistics, planning of experiments and simulation techniques.



Ignatios Deligiannis is an Associate Professor at Alexander Technological Education Institute of Thessaloniki, Greece. His main interests are Object-Oriented software methods, and in particular design assessment and measurement. He received his B.Sc. in Computer Science from Lund University, Sweden, in 1979, and then worked for several years in software development at Siemens Telecommunications industry. He was member of ESERG (Empirical Software Engineering Research Group at Bournemouth University, UK). Currently, he is a research partner of Software Engineering Group::Plase laboratory, Aristotle University of Thessaloniki, Greece.